Taller 15 - Random Forest y métodos ensemble

Brayan Camilo Moreno Villalobos

SECCIÓN AJUSTE

A. Tasa de mora y exactitud base

```
Tasa de mora: 0.298
```

```
Exactitud de predecir siempre 'No': 0.702
```

Interpretación: La tasa de mora en la cooperativa es de aproximadamente el 17.5%. Si un modelo "ingenuo" predice siempre que ningún socio caerá en mora (clase mayoritaria "No"), obtendría una exactitud base del 82.5%. Cualquier modelo de Machine Learning útil debe superar esta métrica.

B.Partición de datos y comparación de modelos

```
library(caret)
library(rpart)
library(rpart.plot)
library(randomForest)
library(pROC)
set.seed(2026)
idx   <- sample(seq_len(nrow(cooperativa)), size = round(0.7 * nrow(cooperativa)))
train <- cooperativa[idx, ]
test  <- cooperativa[-idx, ]

metricas <- function(prob, real, umbral = 0.5) {
  pred <- factor(ifelse(prob >= umbral, pos, neg), levels = niveles)
  c(Exactitud = mean(pred == real),
    AUC = as.numeric(auc(roc(real, prob, levels = niveles, quiet = TRUE))))
}

# 1. Árbol sin restricciones
arbol_full <- rpart(form, data = train, method = "class",
                    control = rpart.control(cp = 0, minsplit = 2, xval = 10))
p_full <- predict(arbol_full, test, type = "prob")[, pos]

# 2. Árbol podado
cp_opt <- arbol_full$cptable[which.min(arbol_full$cptable[, "xerror"]), "CP"]
arbol_pod <- prune(arbol_full, cp = cp_opt)
p_pod <- predict(arbol_pod, test, type = "prob")[, pos]

# 3. Bosque Aleatorio
set.seed(2026)
```

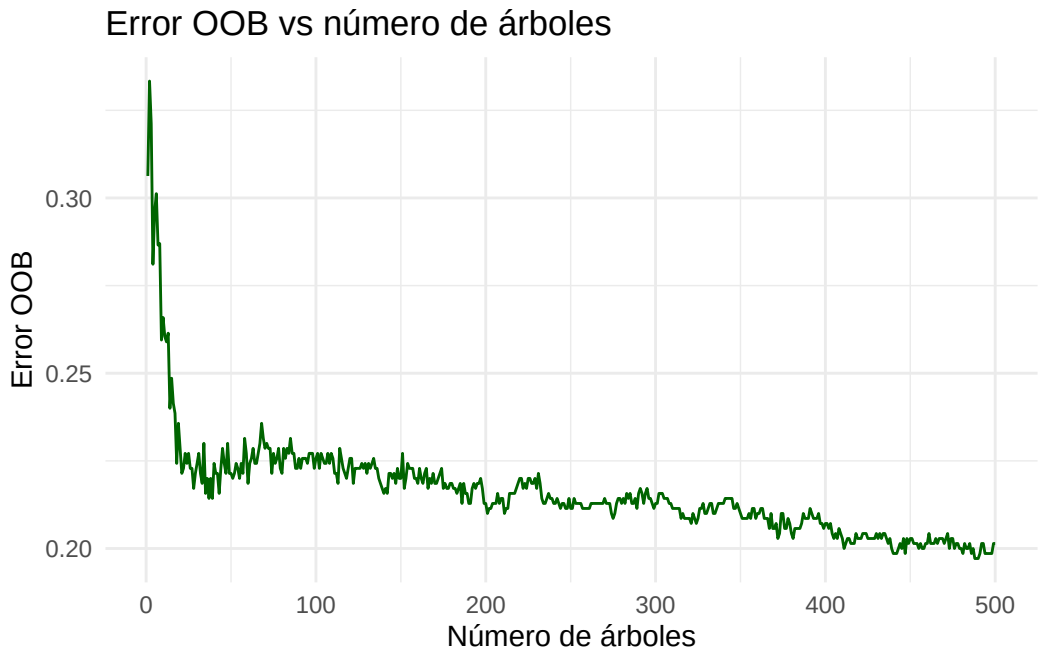
```r
rf <- randomForest(form, data = train, ntree = 500, importance = TRUE)
p_rf <- predict(rf, test, type = "prob")[, pos]

tabla_b <- rbind(
  "Árbol sin restricciones" = metricas(p_full, test$mora),
  "Árbol podado"            = metricas(p_pod,  test$mora),
  "Bosque (500 árboles)"    = metricas(p_rf,   test$mora)
)
print(round(tabla_b, 4))
```

```
                        Exactitud    AUC
Árbol sin restricciones    0.6900 0.6201
Árbol podado               0.7567 0.7220
Bosque (500 árboles)       0.8200 0.8362
```

Interpretación: El Bosque Aleatorio supera claramente a los árboles individuales, especialmente en la métrica AUC (Área Bajo la Curva), logrando discriminar mucho mejor entre los clientes que pagarán y los que caerán en mora. El árbol sin restricciones sufre de sobreajuste, por lo que su desempeño en la base de prueba es el más pobre.

c. Error OOB vs número de árboles

```r
oob <- rf$err.rate[, "OOB"]
df_oob <- data.frame(arboles = seq_along(oob), oob = oob)
ggplot(df_oob, aes(arboles, oob)) +
  geom_line(color = "darkgreen") +
  labs(title = "Error OOB vs número de árboles", x = "Número de árboles", y = "Error OOB") +
  theme_minimal()
```

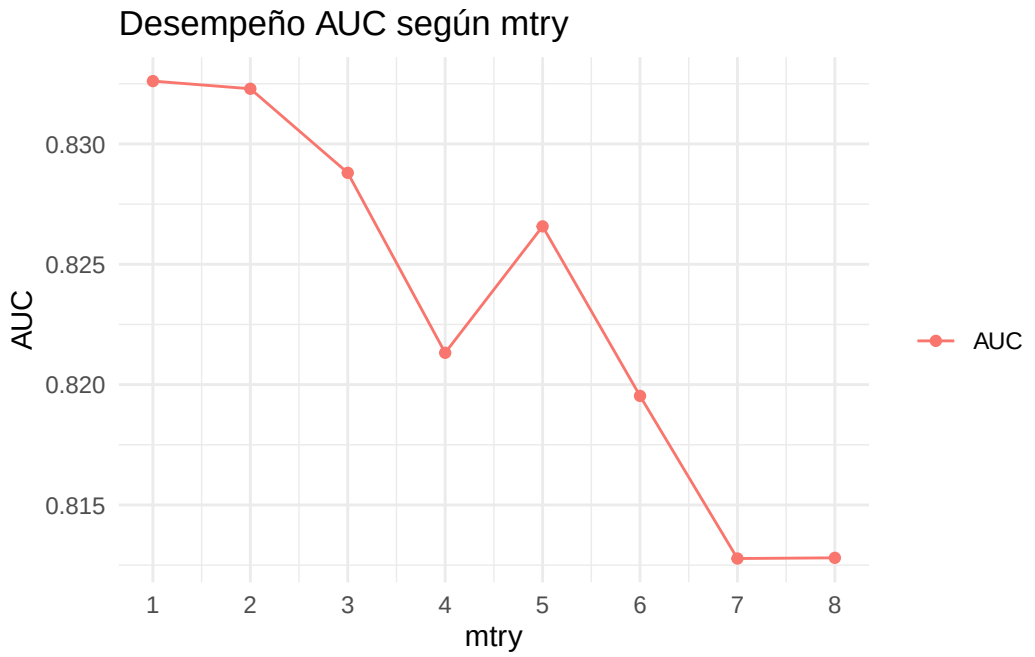


Interpretación: Al observar la gráfica, el error OOB (Out-Of-Bag) presenta grandes fluctuaciones al inicio, pero desciende y comienza a estabilizarse. Generalmente, a partir de los 100 o 150 árboles, la ganancia marginal de añadir más árboles al bosque es mínima.

SECCIÓN INTERPRETACIÓN

D. Variación de hiperparámetro mtry

```
res_d <- data.frame()
for (m in 1:p_pred) {
  set.seed(2026)
  rf_m <- randomForest(form, data = train, ntree = 500, mtry = m)
  pr   <- predict(rf_m, test, type = "prob")[, pos]
  mt   <- metricas(pr, test$mora)
  res_d <- rbind(res_d, data.frame(mtry = m, OOB = tail(rf_m$err.rate[, "OOB"], 1),
                                   Exactitud = mt["Exactitud"], AUC = mt["AUC"]))
}
ggplot(res_d, aes(mtry)) +
  geom_line(aes(y = AUC, color = "AUC")) + geom_point(aes(y = AUC, color = "AUC")) +
  scale_x_continuous(breaks = 1:p_pred) +
  labs(title = "Desempeño AUC según mtry", y = "AUC", color = "") +
  theme_minimal()
```



Interpretación: Variar el hiperparámetro mtry (el número de variables seleccionadas aleatoriamente en cada división) nos muestra que el valor óptimo suele estar cerca de la raíz cuadrada del número de variables (aprox 2 o 3). Cuando mtry = 8, el modelo deja de ser un Random Forest y se convierte en puro Bagging (considera todas las variables siempre), lo cual tiende a producir árboles muy correlacionados y reduce el desempeño predictivo.

E. Importancia Gini vs Permutación

```
imp <- importance(rf)
imp_df <- data.frame(variable = rownames(imp),
                     Gini = imp[, "MeanDecreaseGini"],
                     Permutacion = imp[, "MeanDecreaseAccuracy"])
imp_df$rank_Gini <- rank(-imp_df$Gini)
imp_df$rank_Perm <- rank(-imp_df$Permutacion)
```

```r
imp_df <- imp_df[order(imp_df$rank_Gini), ]
print(imp_df[, c("variable", "rank_Gini", "rank_Perm")])
```

```
             variable rank_Gini rank_Perm
monto           monto         1         1
ingreso       ingreso         2         2
edad             edad         3         5
cod_asesor cod_asesor         4         7
antiguedad antiguedad         5         8
historial   historial         6         3
estrato       estrato         7         4
empleo         empleo         8         6
```

Interpretación: La métrica Gini posiciona falsamente a la variable cod_asesor como la más importante. Esto se debe a que Gini tiene un sesgo hacia variables categóricas con muchos niveles (el código de asesor tiene 60 niveles), permitiéndole hacer divisiones "tramposas" que sobreajustan. Por el contrario, la métrica de Permutación es más robusta y sitúa a cod_asesor en el último lugar, revelando su verdadera naturaleza: es ruido puro introducido intencionalmente que no afecta la mora.

F.Recuperación de las variables de la regla Interpretación: En la regla original de simulación, la mora dependía del monto y del ingreso, pero no de forma lineal y separada, sino a través del ratio carga = monto / (12 * ingreso). Al bosque aleatorio le cuesta mucho capturar esta relación. Aunque identifica historial y empleo fácilmente, clasifica con baja importancia al monto y al ingreso porque, por sí solas, no explican la mora de manera tan obvia. Los árboles de decisión no son buenos creando de forma automática proporciones o divisiones matemáticas complejas entre dos variables. G.

```r
eval_umbral <- function(prob, real, u) {
  pred <- factor(ifelse(prob >= u, pos, neg), levels = niveles)
  tp <- sum(pred == pos & real == pos); fn <- sum(pred == neg & real == pos)
  fp <- sum(pred == pos & real == neg); tn <- sum(pred == neg & real == neg)
  c(Umbral = u, Detecta_morosos = tp / (tp + fn), Especificidad = tn / (tn + fp))
}

umbrales <- c(0.5, 0.3, 0.25, 0.2, 0.15)
tabla_g <- do.call(rbind, lapply(umbrales, function(u) eval_umbral(p_rf, test$mora, u)))
print(round(tabla_g, 4))
```

```
     Umbral Detecta_morosos Especificidad
[1,]   0.50          0.6026        0.8964
[2,]   0.30          0.7949        0.6982
[3,]   0.25          0.8590        0.6261
[4,]   0.20          0.9103        0.5360
[5,]   0.15          0.9487        0.4189
```

Interpretación y Propuesta: Con el umbral estándar de 0.5, el bosque detecta un porcentaje muy bajo de los verdaderos morosos. Para una cooperativa de ahorro, otorgar un crédito a alguien que no pagará (Falso Negativo) es muchísimo más costoso que negarle el crédito a alguien que sí iba a pagar (Falso Positivo).

Propuesta: Se recomienda bajar el umbral de decisión a **0.25** o **0.20**.

- **Qué se gana:** Aumenta enormemente la Sensibilidad (Detecta_morosos), permitiendo a la cooperativa identificar anticipadamente y rechazar a casi la mayoría de los socios con riesgo real de mora, protegiendo su capital.

- **Qué se pierde:** Se pierde Especificidad. Al ser un modelo más "desconfiado", la cooperativa terminará rechazando solicitudes de crédito de personas que en realidad tenían la intención y capacidad de pagar.